Experiment: Measuring Process Execution Time Using /proc Filesystem

To write a simple C program that prints the system time, execute it, and then use the /proc filesystem to determine how long the process ran in **user mode** and **kernel mode**.

Explanation

When a program runs on Linux, the CPU executes its instructions in two modes:

- **User mode:** executing code in user space (e.g., arithmetic, loops)
- **Kernel mode:** executing system calls (e.g., file I/O, printing)

Linux maintains process statistics in /proc/[pid]/stat, which includes:

- **utime** – Time spent in user mode (in clock ticks)
- **stime** – Time spent in kernel mode (in clock ticks)

By examining these fields after the program finishes, we can estimate how long it ran in each mode.

🧪 Step 1: Write the C Program

Create a file named time_display.c:

```
#include <stdio.h>

#include <time.h>

#include <unistd.h>

int main() {

   time_t now; //time_t is a data type used to represent calendar time.

   time(&now);  //Stores the current calendar time (seconds since the Unix Epoch) into the variable now.

   printf("Current system time: %s", ctime(&now));  // Converts the calendar time stored in now into a human-readable string.

sleep(300);

   return 0;

}
```

🧪 Step 2: Compile and Run the Program

```
gcc time_display.c -o time_display

./time_display
```

Sample Output:

Current system time: Mon Oct 28 11:37:21 2025

🔍 Step 3: Find the Process ID (PID)

You can run the program in the background to capture its PID:

./time_display &

Sample Output:

[1] 4512

Current system time: Mon Oct 28 11:37:25 2025

Here, the **PID** of the process is 4512.

🧩 Step 4: Check Process Statistics from /proc

While the process is still running (or immediately after starting it), view its status using:

cat /proc/4512/stat

This outputs a long line with many space-separated fields.

Example (shortened):

4512 (time_display) R 4498 4512 4498 0 -1 4194560 186 0 0 0 0 1 0 0 20 0 1 0 123456 123456 0 0 0 0 0 0 0 0 0 0 0 0

R-process state, 4498 – PPID, 4512 PID,4498 -PgroupID, 4194560 kernel flag,01 usermode kernel mode cpu time,20 priority, 123456 virtual memory size in bytes, 123456 pages ...

🧩 Step 5: Identify the utime and stime Fields

According to the Linux documentation (man proc), the fields are:

FieldDescription

14 **utime** – Time spent in user mode (in clock ticks)

15 **stime** – Time spent in kernel mode (in clock ticks)

To extract just these values:

awk '{print "utime (user mode): "$14, "\nstime (kernel mode): "$15}' /proc/4512/stat

Sample Output:

utime (user mode): 2

stime (kernel mode): 1

🧩 Step 6: Convert Clock Ticks to Seconds

Linux measures CPU time in **clock ticks**, which vary by system.
To find the number of clock ticks per second:

getconf CLK_TCK

Typical output:

100

This means **100 ticks = 1 second**.

Now convert manually:

- User time = 2 / 100 = **0.02 seconds**
- Kernel time = 1 / 100 = **0.01 seconds**

So the process ran for about **0.03 seconds total**.

🧩 Step 7: Verify with /proc/[pid]/status (Alternative)

You can also read:

cat /proc/4512/status | grep -E "utime|stime"

However, note that in many systems these fields are not shown directly here — /proc/[pid]/stat is more reliable for precise values.

awk '{print $14, $15}' /proc/4512/stat

🧩 Step 8: Explanation

Field	Meaning
utime	CPU time spent executing user instructions
stime	CPU time spent in system calls or kernel code
CLK_TCK	System constant defining ticks per second